

开源软件基础

基于 z3 的约束求解

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

November 8, 2020



Outline

① 开始之前

② z3 简介

③ 示例

课堂效果的反思

不足之处

- ① 没有考虑到系统的差异
- ② 节奏还是偏快

采取措施

- ① 代码在 windows 系统下进行测试
- ② 课堂演示也使用 windows 系统
- ③ 下周课程安排，根据调查结果调整

希望我们还是朋友

课堂效果的反思

不足之处

- ① 没有考虑到系统的差异
- ② 节奏还是偏快

采取措施

- ① 代码在 windows 系统下进行测试
- ② 课堂演示也使用 windows 系统
- ③ 下周课程安排，根据调查结果调整

希望我们还是朋友

Outline

① 开始之前

② z3 简介

③ 示例

z3 简介

概念

Z3 is a theorem prover from Microsoft Research. It is licensed under the MIT license^a.

^a<https://github.com/Z3Prover/z3>

约束求解的历史

- ① SAT: The satisfiability problem
- ② SMT: Satisfiability modulo theories

可满足问题

SAT

可满足性 (Satisfiability) 是用来解决给定的真值方程式，是否存在一组变量赋值，使问题为可满足。布尔可满足性问题 (Boolean satisfiability problem; SAT) 属于决定性问题，也是第一个被证明属于 NP 完全的问题。此问题在计算机科学上许多领域的皆相当重要，包括计算机科学基础理论、算法、人工智能、硬件设计等等。^a

^a<https://zh.wikipedia.org/zh-cn/>

Outline

① 开始之前

② z3 简介

③ 示例

约束求解示例

今有雉兔同笼，上有三十五头，下有九十四足，问雉兔各几何？

约束求解示例

上置头，下置足，半其足，以头除足，以足除头，即得。

孙子算经

约束求解示例

```
import z3
chicken, rabbits = z3.Ints('chicken rabbits')
z3.solve(chicken >= 1,      # number of chicken
         rabbits >= 1,      # number of rabbits
         chicken + rabbits == 35,
         chicken * 2 + rabbits * 4 == 94)
```

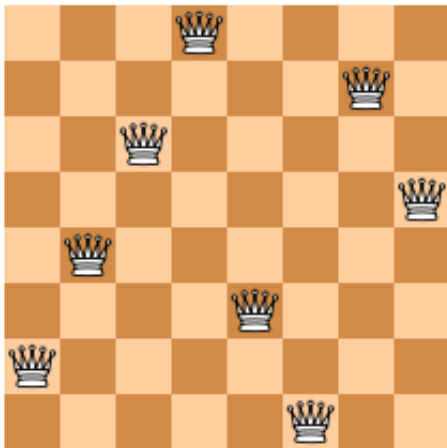
约束求解示例

$$\begin{aligned} \bigcirc + \bigcirc &= 10 \\ \bigcirc \times \square + \square &= 12 \\ \bigcirc \times \square - \triangle \times \bigcirc &= \bigcirc \\ \triangle &= ? \end{aligned}$$

约束求解示例

```
from z3 import *
circle, square, triangle = Ints('circle square triangle')
s = Solver()
s.add(circle+circle==10)
s.add(circle*square+square==12)
s.add(circle*square-triangle*circle==circle)
print(s.check())
print(s.model())
```

八皇后问题



八皇后问题

```
Q = [ Int('Q_%i' % (i + 1)) for i in range(8) ]

# Each queen is in a column {1, ... 8 }
val_c = [ And(1 <= Q[i], Q[i] <= 8) for i in range(8) ]

# At most one queen per column
col_c = [ Distinct(Q) ]

# Diagonal constraint
diag_c = [ If(i == j,
             True,
             And(Q[i] - Q[j] != i - j, Q[i] - Q[j] != j - i))
          for i in range(8) for j in range(i) ]

solve(val_c + col_c + diag_c)
```

程序生成/修复

```
from z3 import *
x = Int('x')
y = Int('y')
f = Function('f', IntSort(), IntSort())
s = Solver()
s.add(f(f(x)) == x, f(x) == y, x != y)
print(s.check())
m = s.model()
print("f(f(x)) =", m.evaluate(f(f(x))))
print("f(x)      =", m.evaluate(f(x)))
```